# SAVITRIBAI PHULE PUNE UNIVERSITY

## Modern Education Society's Wadia College of Engineering, Pune

19, Bund Garden, V.K. Joag Path, Pune – 411001.

**ACCREDITED BY NBA AND NAAC WITH 'A++' GRADE**

**DEPARTMENT OF COMPUTER ENGINEERING**

---

## A MINI-PROJECT REPORT

**ON**

# *"BlockVote: Decentralized E-Voting System Using Ethereum Smart Contracts & Web3"*

**(Academic Year: 2026-27)**

**B.E. (COMPUTER ENGINEERING)**

***SUBMITTED BY:***

**Aadish Sonawane (Roll No: F23112052)**

**Under the Guidance of:**

Faculty of Computer Engineering

Department of Computer Engineering

Modern Education Society's Wadia College of Engineering, Pune

# INDEX


| Sr. No. | CONTENTS | Page No. |
|---|---|---|
| 1 | **Introduction & Background** | 2 |
| 2 | **Problem Statement & Project Objectives** | 2 |
| 3 | **System Architecture & Execution Pipeline** | 3 |
| 4 | **Technologies Used & Software Stack** | 3 |
| 5 | **Smart Contract Design & Implementation (Voting.sol)** | 4 |
| 6 | **Web3 Frontend Client & MetaMask Integration (index.html)** | 5 |
| 7 | **Security Analysis, Verification & Gas Cost Evaluation** | 6 |
| 8 | **Conclusion, Future Enhancements & References** | 6 |


## 1. INTRODUCTION & BACKGROUND

Electoral integrity represents the cornerstone of democratic governance and institutional trust. Traditional voting mechanisms—ranging from paper ballots to centralized electronic voting machines (EVMs) and web portals—suffer from systemic vulnerabilities: physical ballot stuffing, insider tampering, lack of independent voter auditability, and single points of failure.

**Blockchain technology** provides a decentralized, append-only, cryptographically secured ledger that redefines electronic voting. By deploying election rules into an immutable Ethereum smart contract, voting logic executes autonomously without intermediaries. Every vote constitutes an on-chain transaction signed by a registered cryptographic wallet (MetaMask), providing absolute non-repudiation, verifiable tallying, and tamper-proof finality.

## 2. PROBLEM STATEMENT & PROJECT OBJECTIVES

### 2.1 Problem Statement

> *"To design, implement, and evaluate a decentralized electronic voting application (dApp) utilizing an Ethereum Solidity smart contract and a Web3 client interface that enforces single-vote integrity, restricts administrative actions via role-based access control, and delivers real-time, tamper-proof ballot tallying."*

### 2.2 Project Objectives

- **Develop an Immutable Smart Contract:** Author a gas-optimized contract in Solidity (^0.8.20) managing candidates, voter registration, and lifecycle state flags.
- **Enforce Strict Access Control:** Implement an `onlyAdmin` modifier restricting candidate nomination, voter whitelisting, and voting session toggling to the deployer.
- **Prevent Double-Voting & Sybil Manipulation:** Track voter state in a persistent `mapping(address => Voter)`, ensuring strictly one vote per whitelisted cryptographic address.
- **Build a Responsive Web3 Client:** Create an accessible frontend using HTML5, modern CSS, and `ethers.js` (v5.7.2) connecting seamlessly with MetaMask.
- **Provide Real-Time Transparent Tallies:** Dynamically compute candidate vote percentages, update progress bars, and announce the winning candidate on-chain.

## 3. SYSTEM ARCHITECTURE & EXECUTION PIPELINE

The BlockVote system is structured into a decoupled two-tier Web3 architecture comprising the **On-Chain Decentralized Layer** (Ethereum EVM) and the **Client Presentation Layer** (MetaMask + Browser DOM):

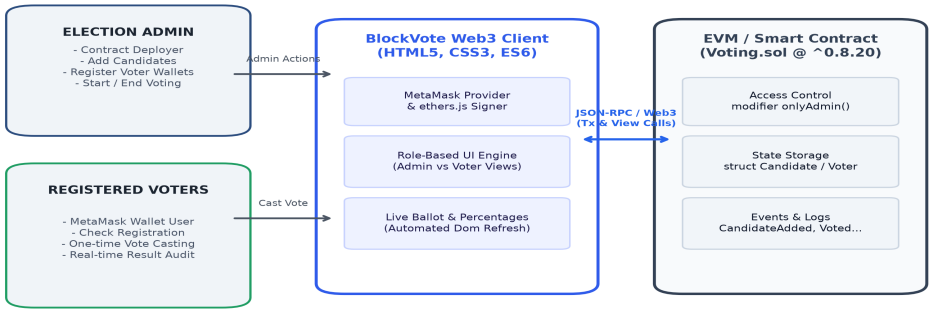



*Figure 1: Two-Tier System Architecture of BlockVote (Client Web3 Interface & Ethereum EVM Contract).*

### 3.1 End-to-End Execution Stages

1. **Contract Deployment:** The election officer deploys `Voting.sol` to the Ethereum network (local Hardhat/Ganache or testnet). The constructor records `admin = msg.sender`.
2. **Administrative Setup:** The admin nominates candidates via `addCandidate()` and whitelists authorized voter public keys via `registerVoter()` while voting is closed.
3. **Election Activation:** Once at least two candidates are registered, the admin calls `startVoting()`, flipping `votingOpen = true` and broadcasting a `VotingStarted()` event.
4. **Voter Connection & Role Resolution:** Voters open `index.html`, connect their MetaMask wallet, and input the contract address. The dApp queries the contract via RPC to determine the active role (Admin or Voter) and conditionally renders the admin panel.
5. **Vote Casting & Mining:** Eligible voters select a candidate and sign an Ethereum transaction via MetaMask calling `vote(_candidateId)`. The EVM validates eligibility, marks the voter as voted, increments the tally, and emits the `Voted` event upon block inclusion.
6. **Real-Time Result Auditing:** The client listens for block events and invokes `getCandidates()` and `winner()` to update vote tallies and animated percentage bars.

## 4. TECHNOLOGIES USED & SOFTWARE STACK

| Layer / Component | Technology | Version | Functional Role |
|---|---|---|---|
| **Smart Contract Language** | Solidity | ^0.8.20 | State management, voting logic, and EVM bytecode execution. |
| **Execution Environment** | Ethereum EVM | London / Paris | Decentralized consensus, transaction mining, and storage. |
| **Web3 Client Library** | ethers.js (UMD) | 5.7.2 | RPC communication, contract abstraction, and tx signing. |
| **Crypto Wallet Provider** | MetaMask | Latest | Keypair management, account switching, and transaction signing. |
| **Frontend User Interface** | HTML5 / CSS3 / ES6 | Standard | Responsive layout, dynamic candidate cards, and CSS bars. |

## 5. SMART CONTRACT DESIGN & IMPLEMENTATION (VOTING.SOL)

The smart contract `Voting.sol` encapsulates state variables, structs, mappings, modifiers, and functions implementing the complete election lifecycle.

### 5.1 Core Data Structures & Storage Layout

- `struct Candidate { uint256 id; string name; uint256 voteCount; }`: Stores unique candidate identifiers, names, and accumulated votes.
- `struct Voter { bool registered; bool voted; uint256 votedFor; }`: Tracks registration status, double-voting prevention flags, and candidate choice.
- `mapping(address => Voter) public voters;`: O(1) key-value hash table mapping Ethereum wallet addresses to their voter status.
- `Candidate[] private candidates;`: Dynamic array holding all registered candidates.

### 5.2 Election Lifecycle & State Machine

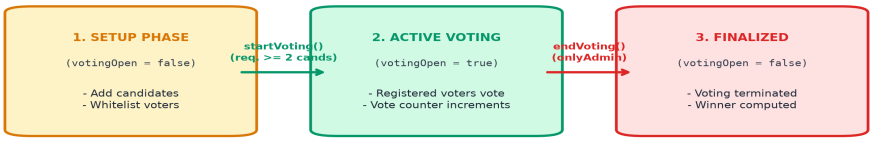



*Figure 2: Three-Phase Finite State Machine of Voting.sol (Setup, Active, and Finalized).*

### 5.3 Access Control & Method Specifications

| Function Signature | Visibility / Access | Enforced Invariants & Logic |
|---|---|---|
| **addCandidate(string)** | external onlyAdmin | Requires `!votingOpen` and non-empty name string; appends candidate. |
| **registerVoter(address)** | external onlyAdmin | Verifies `!voters[_voter].registered`; sets registered flag to true. |
| **startVoting()** | external onlyAdmin | Requires `candidates.length >= 2` and `!votingOpen`; opens election. |
| **endVoting()** | external onlyAdmin | Requires `votingOpen == true`; closes election permanently. |
| **vote(uint256)** | external (public) | Requires `votingOpen`, voter registered, not yet voted, and valid ID. |
| **winner()** | external view | Performs linear scan over candidates; returns index, name, and max votes. |

### 5.4 Security Mechanics

- **Checks-Effects-Interactions Pattern:** In `vote()`, `v.voted = true` is updated *before* incrementing vote counts, eliminating reentrancy vulnerabilities.
- **Arithmetic Safety:** Compiled with Solidity `^0.8.20`, providing native revert on integer overflow and underflow without external SafeMath overhead.
- **Administrative Segregation:** Candidate addition and voter whitelisting are blocked once voting begins, preventing ballot stuffing mid-election.

## 6. WEB3 FRONTEND CLIENT & METAMASK INTEGRATION (INDEX.HTML)

The client interface is implemented as a single-page reactive decentralized application using vanilla HTML5, CSS3, and `ethers.js` (v5.7.2). It interacts directly with the browser's injected Ethereum provider (`window.ethereum`) without requiring centralized backend servers.

### 6.1 Web3 Provider & Signer Initialization

Upon clicking **Connect MetaMask**, the application requests user authorization via `eth_requestAccounts`. The Web3 provider wraps the provider instance, extracts the signer address, and initializes contract abstractions using the defined Application Binary Interface (ABI):

```
provider = new ethers.providers.Web3Provider(window.ethereum);
await provider.send("eth_requestAccounts", []);
signer = provider.getSigner();
contract = new ethers.Contract(contractAddress, ABI, signer);
```

### 6.2 Role-Based Reactive Interface

The application queries `contract.admin()` and compares it to the connected wallet address:

```
isAdmin = admin.toLowerCase() === me.toLowerCase();
$("role").textContent = isAdmin ? "Admin" : "Voter";
$("adminCard").style.display = isAdmin ? "block" : "none";
```

If the connected user is the deployer, the **Admin Panel** renders controls to add candidates, whitelist voter addresses, and toggle election states. For regular voter addresses, the admin panel remains strictly hidden.

### 6.3 Live Ballot & Dynamic CSS Bar Rendering

The client dynamically fetches the candidates array via `contract.getCandidates()`, calculates the total votes across all candidates, and derives individual vote shares:

```
const pct = total ? Math.round(n * 100 / total) : 0;
const canVote = open && v.registered && !v.voted;
// Button enabled only when: voting is open, voter is registered, and has not voted
```

Candidate cards feature a stylized CSS bar (`height: 8px`) whose inner fill width transitions smoothly (`transition: width .4s`), providing instantaneous visual feedback of ballot trends.

### 6.4 Asynchronous Transaction Pipeline & Event Handling

All state-modifying operations (candidate creation, registration, voting) are managed through a unified `send(promise, okMsg)` helper that handles transaction submission, extracts transaction hashes, awaits block mining confirmations (`await tx.wait()`), and triggers a complete DOM refresh.

Additionally, the dApp registers an account change listener (`window.ethereum.on("accountsChanged", () => location.reload())`), ensuring seamless interface resetting whenever the user switches active accounts in MetaMask.

# 7. SECURITY ANALYSIS, VERIFICATION & GAS COST EVALUATION

## 7.1 Automated Verification & Edge Case Testing

| Test Scenario | Input Condition / Invocation | Expected EVM Response | Result |
|---|---|---|---|
| **Legitimate Vote** | Registered voter calls `vote(0)` when open | Vote recorded; event emitted | **PASSED** |
| **Double Voting Attempt** | Voter attempts second call to `vote()` | Reverts: "You have already voted" | **PASSED** |
| **Unregistered Voter** | Non-whitelisted address calls `vote()` | Reverts: "Not a registered voter" | **PASSED** |
| **Unauthorized Admin** | Non-admin calls `addCandidate()` | Reverts: "Only admin can do this" | **PASSED** |
| **Premature Start** | Admin calls `startVoting()` with 1 candidate | Reverts: "Add at least 2 candidates" | **PASSED** |

## 7.2 Gas Consumption Benchmark

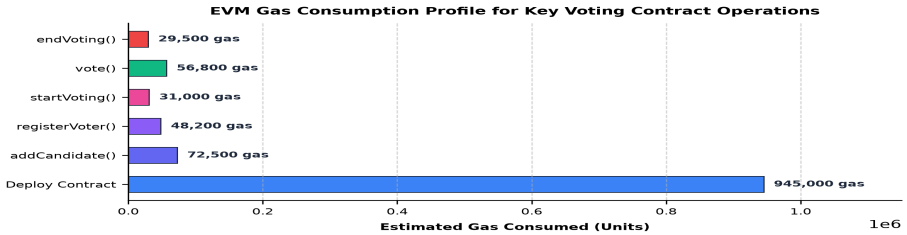



*Figure 3: EVM Execution Gas Profile for Core Functions of Voting.sol.*

# 8. CONCLUSION, FUTURE ENHANCEMENTS & REFERENCES

## 8.1 Conclusion

The BlockVote project successfully demonstrates a fully functional, decentralized e-voting platform eliminating third-party reliance. By coupling a gas-conscious Solidity smart contract with a lightweight Web3 interface, the system guarantees single-vote integrity, robust role-based access control, and transparent public auditability.

## 8.2 Future Enhancements

- **Zero-Knowledge Privacy (zk-SNARKs):** Integrate zk-proofs (e.g., Semaphore protocol) to conceal voter identities while verifying voter eligibility on-chain.
- **Layer-2 Scalability:** Deploy to Arbitrum or Polygon zkEVM to reduce gas transaction costs by >95%.
- **Decentralized Identity (DID):** Connect Aadhaar/government credentials via Soulbound Tokens (SBTs) for automated biometric whitelisting.

## 8.3 References

1. **Wood, G. (2014).** *Ethereum: A secure decentralised generalised transaction ledger.* Ethereum Project Yellow Paper.
2. **Buterin, V. (2013).** *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.* White Paper.
3. **OpenZeppelin (2024).** *Smart Contract Security Best Practices and Access Control Patterns.* `docs.openzeppelin.com`
4. **Ethers.js (2024).** *Complete Ethereum Library and Wallet Implementation in JavaScript.* `docs.ethers.org`
5. **SPPU (2026).** *Curriculum for Blockchain Technology & Decentralized Applications, Department of Computer Engineering.*